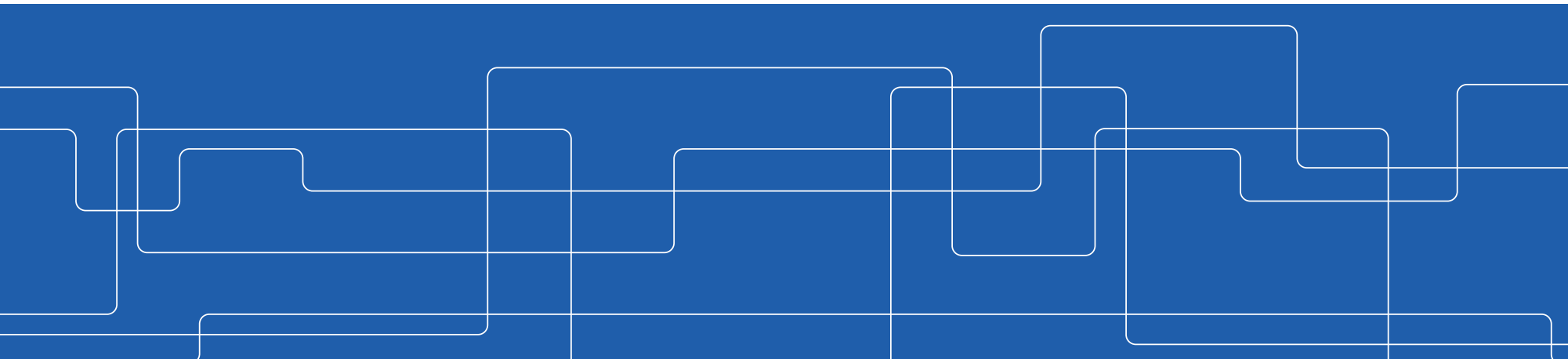


Search in Games

Artificial Intelligence (DD2380)

KTH – Royal Institute of Technology





Zero-sum games

- **Utility function** $\psi(s, p)$: numerical value for a game ending at state s by player p.
- **Zero-sum game**: game where the sum of the utility function for all players is zero. That is: $\sum_i \psi(s, p_i) = 0$.
  - Chess example: $\psi(\texttt{win}, p) = 1$, $\psi(\texttt{loss}, p) = -1$, and $\psi(\texttt{draw}, p) = 0$.
- We consider **2-player**, **adversarial** games (no collaboration).
- We consider **perfect information** games: **deterministic** and **fully observable** environments.

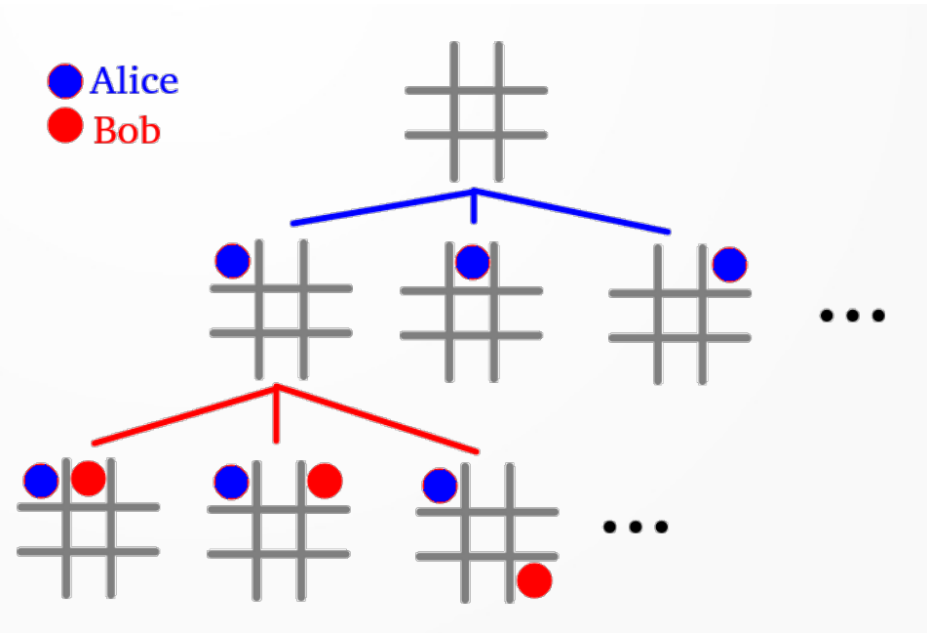
Zero-sum games

Tree representation:

- Nodes are states.
- Edges are moves (actions).

Optimizing next move:

- Exploring moves and their affects to some depth.
- Assume the opponent is acting optimally according to your knowledge.
- Search for the move that reaches the best leaf state.



Mini-Max Algorithm

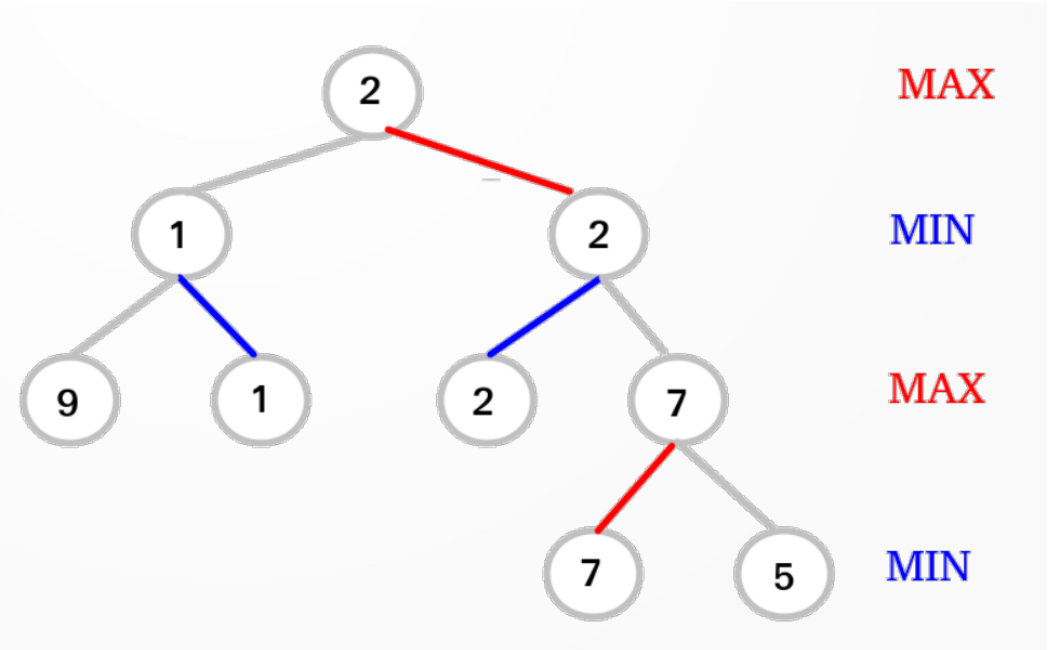
- Strategy to find the best move.
- Two players taking turns: `MAX` and `MIN`.
  - `MAX` wants to **maximize** their gain.
  - `MIN` wants to **minimize** the other's gain.
- We want to explore the game tree to find the best outcome: $\rightarrow$ **Search!**
  - Depth First Search (DFS).

Mini-Max Algorithm

Two player types:

- `MAX`: selects highest value state.
- `MIN`: selects lowest value state.

We assume both players are playing optimally based on our knowledge.





Alpha-Beta Pruning

- Allows us to skip exploring parts of the tree, improving performance.
- When we learn that we are exploring a branch that one of the players would never choose to play towards, we stop exploring that branch.
- We achieve this by keeping two variables **Alpha** and **Beta**.

Alpha-Beta Pruning

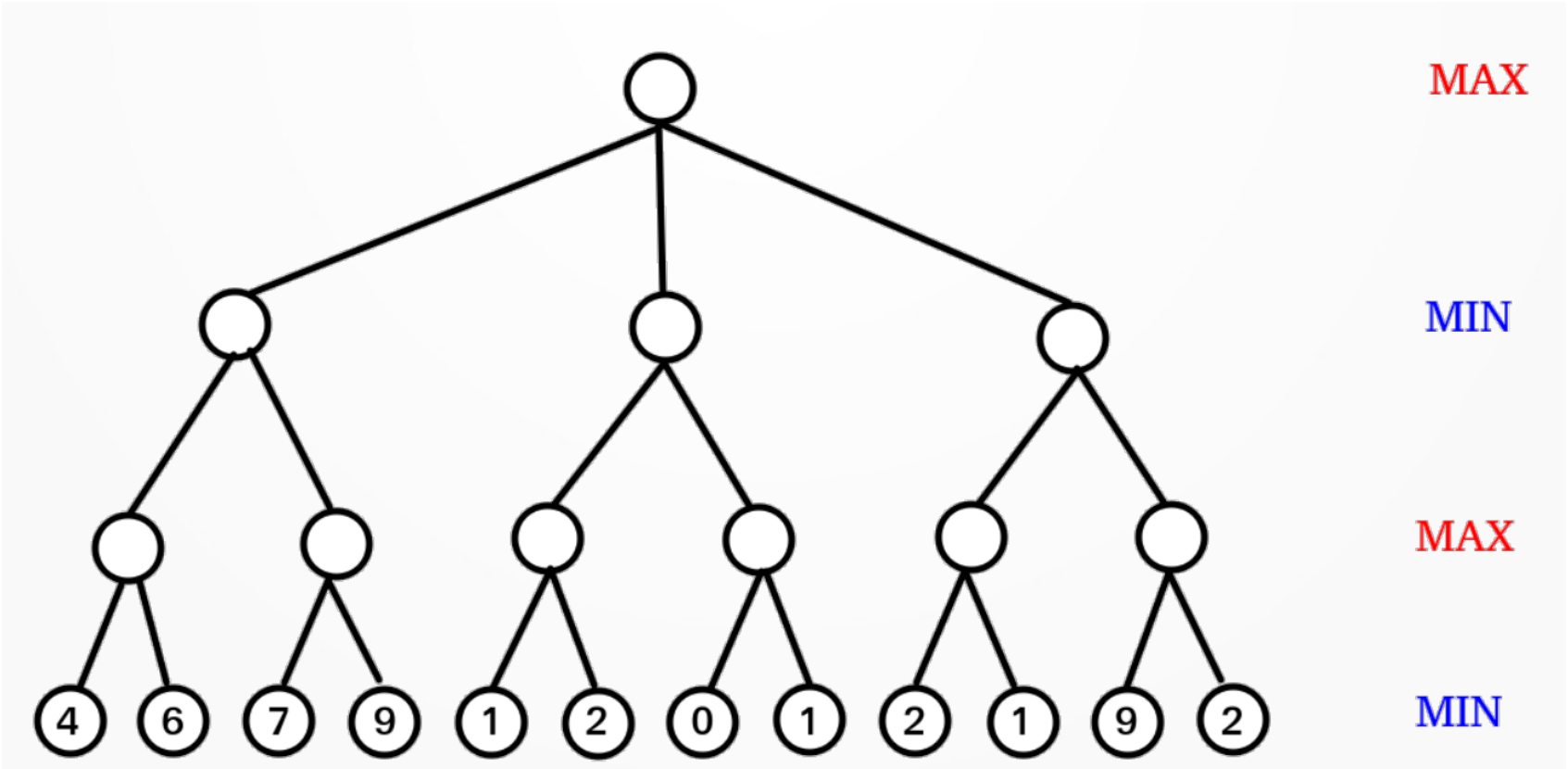
- **Alpha**: Best value found along the path to the root for MAX
  - At some point on the path to the current node the MAX player has the choice to force us down a branch were they are guaranteed the α value.
- **Beta**: Best value found along the path to the root for MIN
  - At some point on the path to the current node the MIN player has the choice to force us down a branch were they are guaranteed the β value.
- Initialized as $\alpha = -\infty$, $\beta = \infty$
- α and β values are only propagated down the tree.
- α values are updated in MAX nodes.
- β values are updated in MIN nodes.



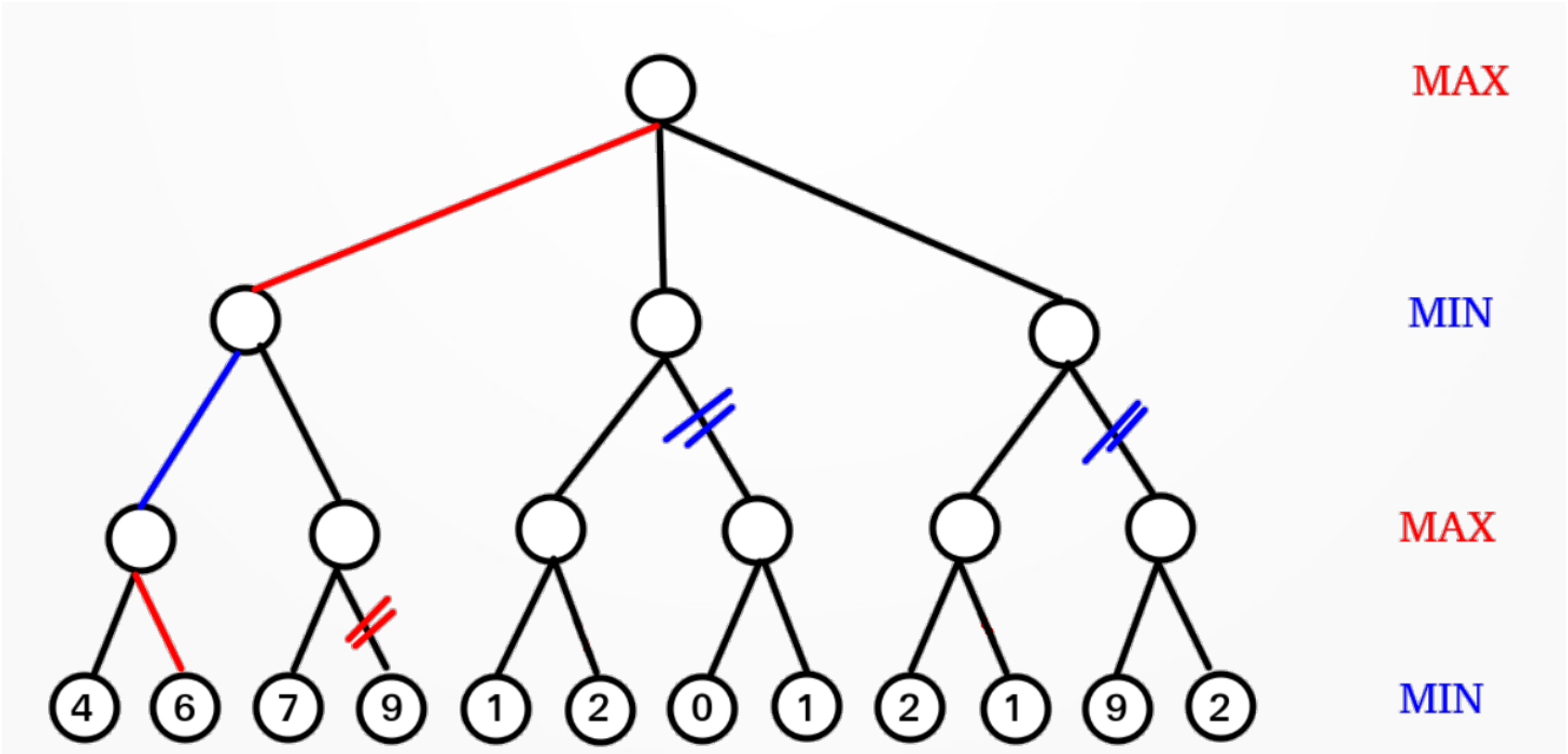
Alpha-Beta Pruning

- We prune when $\beta \leq \alpha$.
- At a `MAX` node it means that the `MAX` player has found a value that is worse for the `MIN` player than what `MIN` is guaranteed by forcing the game down a different branch prior to reaching the current node.
  - Given that `MIN` plays optimally the game will never reach the current node and therefore there is no need to explore it further.
- Similarly at a `MIN` node but the roles are reversed.

Alpha-Beta Pruning



Alpha-Beta Pruning





Heuristic

- It's not always possible to explore a game until it's end. Therefore we utilize a heuristic function to approximate the utility function (the true value of a state)
- In a zero-sum game a state which is beneficial for the `MAX` player should produce a positive heuristic value. And a state which is beneficial for the `MIN` player should produce a negative heuristic value.
- The heuristic should not be too computationally expensive.



Iterative Deepening Search (IDS)

- In many games it's not possible to search until the end of the game.
- We could search to a fixed depth. But if it's too shallow we could have gained more information. And if it's too deep we will only be able to explore a fraction of the moves.
- We can solve this by iteratively increasing the search depth. Performing a full search for each depth until the resources run out.
  $0 \rightarrow d, 0 \rightarrow d + 1, \ldots$



Repeated States

- Search algorithms can visit the same state via different sequences of moves.
- We can implement a data structure to avoid calculating the same states and thereby improve performance. $\rightarrow$ **Hash function** and **transposition table**.
- Which states are equivalent?
  - Same placement of pieces.
  - **Symmetric** placements.
  - Is it important **who** made the move?



Move ordering

- We can improve the amount of the tree that **AlphaBeta** will prune by implementing move ordering.
- The best move for the respective players should be explored first.
- Moves can be ordered based on:
  - Heuristic- or transposition table values
  - Results from previous IDS



Tips and Questions

1. Make sure your **MiniMax** works.
2. Make sure your **Alpha-Beta** pruning works.
3. Make sure your **heuristic** works as intended.
4. Implement **iterative deepening search**.
5. Implement **move ordering**.
6. Check for **repeated states**.
7. Explore other methods
   - End state look-up table
   - Killer moves
   - ...



Tips and Questions

- Questions?
- **Have fun! :)**